Lumen Critical Path — shipping Lumen on iOS + Android

> **The single litmus test** (`00-roadmap.md:66`): *Lumen — pick → restore → before/after compare → paywall → save → share — runs natively on a real iPhone and a real Android device, ships to TestFlight and the Play Store, and passes the same E2E spec on both.* This document re-prioritizes the master plan (`10-competitor-master-plan.md`) around that one goal. Where the master plan asks "what makes a good RN competitor," this asks "what is the shortest correct path to Lumen on both stores." Build top-to-bottom. *Status date 2026-06-15. Grounded in `examples/lumen-probe/src/Main.can` (the live capability probe) + the app at `/home/quinten/projects/apps/lumen` . Supersedes the stale 2026-06-12 gap analysis in the Lumen repo, which predates the Phase-4 capability build-out.*

---

0. Where we actually are (honest)

Android: ~feature-complete for the Lumen flow, minus one screen and the release/asset plumbing. The `lumen-probe` app validates **on-device** every capability the litmus flow needs *except* the before/after compositor:

Probe gate	Capability	Android
decode bundled photo → `CanopyBitmap`	`canopy/image`	✅ on device
Restore → ESPCN ONNX inference	`canopy/inference` (RestoreEngine)	✅ on device
Pick → Android Photo Picker → decode	`canopy/photos`	✅ on device
Save → MediaStore	`canopy/album`	✅ on device
Share → system share sheet	`canopy/share-image`	✅ on device
Store → EncryptedSharedPreferences	`canopy/storage-secure`	✅ on device
Notify → notification	`canopy/notify`	✅ on device
Before/After wipe compositor	`Native.BeforeAfter`	❌ **disabled — host-layout collapse bug**
Paywall (one-time unlock)	`canopy/billing` (Play Billing)	◐ module exists; not yet wired into a Lumen paywall, not store-verified

So the *entire* Lumen data/UI/ML path runs on Android today. The two genuine gaps are (1) the **before/after compositor** — `examples/lumen-probe/src/Main.can:206-213` disables it: *"the native LumenBeforeAfterView + wipe gesture are built + wired, but it collapses its host-layout parent (undiagnosed C2 fidelity issue)"* — and (2) the **release/asset/store pipeline** (signed AAB, R8 keep-rules, packaging the ~59 MB ONNX-runtime `.so` , billing entitlement).

iOS: 0% — never compiled. ~5,571 lines of Objective-C++ host exist (incl. `CanopyRestoreEngineModule.mm` for Core ML, all capability `.mm` modules) but no `xcodebuild` has ever run. Every Lumen step on iOS is unvalidated. The remote Mac harness (`host/ios/remote-build.sh` , now with `provision`) is the path; **first compile is scheduled for today.**

Just landed (this session):

- **Release security** — `MainActivity.readBundle()` now reads the `/data/local/tmp` override only under `BuildConfig.DEBUG` , and `verifyBundleIntegrity` fails *closed* in release. A shipped Lumen APK has no unsigned code-load path (App Store 2.5.2 / Play review). *(master plan AND-1/DEV-1 — done.)*
- **`lazy` memoization** — `native.js` now short-circuits unchanged `lazy` /thunk subtrees (port of `virtual-dom.js:2216`), with a regression test `harness/run-lazy.js` (in the CI gate). This is what keeps Lumen's batch-queue list and multi-screen re-renders cheap. *(master plan*

RND-1 — done.)

Bottom line: you can start building Lumen's real screens on Android **now**, against a working capability set, using side-by-side thumbnails for compare (as the probe does) until the compositor is fixed. The work below is ordered so Lumen ships on Android first, then iOS reaches parity.

---

1. The litmus flow, step by step

Step	Android	iOS	Gating work
Pick a photo	✅ works	☐ needs iOS bring-up + `CanopyPhotosModule.mm` validated	L-A0 done · L-I path
Restore (super-res)	✅ ESPCN ONNX on device	☐ Core ML / ORT on ANE (`CanopyRestoreEngineModule.mm` , has a known dead weak-symbol)	L-A0 done · **L-I3**
Before/After compare	❌ **compositor collapses layout**	☐ never built on iOS	**L-A1 (top blocker)** → L-I
Paywall (unlock)	◖ Play Billing module exists	☐ StoreKit 2	**L-A4** · **L-I5**
Save to album	✅ MediaStore	☐ PHPhotoLibrary	L-A0 done · L-I path
Share	✅ share sheet	☐ UIActivityViewController	L-A0 done · L-I path

The Android column is green except **compare** and **paywall**; the iOS column is the whole bring-up.

---

2. Lumen critical path (the spine)

```
[LANDED] AND-1 release security · RND-1 lazy memoization
   |
   ▼  ── MILESTONE L-A: "Lumen ships on Android" (signed AAB → Play internal track) ──
 L-A1  Fix Before/After host-layout collapse          (THE top blocker; the screen that markets the app)
 L-A2  Assemble the real Lumen app (apps/lumen) on the live capability set + screens
 L-A3  Signed release: R8 keep-rules for JNI/JSI by-name lookups + assembleRelease boots on device
 L-A4  Wire the paywall: Play Billing entitlement → Storage.Secure, gate restore/export
 L-A5  Asset/size pass: package ORT runtime + model, hit the Play AAB size budget, restore memory budget
 L-A6  E2E: the lumen-restore Appium flow green on the emulator (extend e2e/flows)
   |
   ▼  ── MILESTONE L-I: "Lumen ships on iOS" (TestFlight on a physical iPhone) ──
 L-I1  iOS first compile + Hermes/JSI runtime wired (host/ios via remote-build.sh)   ← STARTS TODAY
 L-I2  Capability parity: validate each .mm module (Photos/Album/Share/Storage/Notify/Image) on device
 L-I3  RestoreEngine on iOS: Core ML (ANE) path; fix the CanopyMakeCoreMLRestoreModule dead weak-symbol
 L-I4  Before/After on iOS (reuse the L-A1 design) + parity test vectors (Android vs iOS hosts)
 L-I5  StoreKit 2 paywall + signing/provisioning + TestFlight build
 L-I6  E2E parity: the SAME lumen-restore spec green on an iPhone (testID → accessibilityIdentifier)
   |
   ▼  ── Both stores ──
```

Everything off this spine (generic component completeness, dev-loop polish beyond Fast Refresh, the broader RN-competitor surface) is deferred to the master plan's own phases.

---

3. Phased work items

Effort in engineer-weeks (ew). "Why" is tied to the Lumen flow. IDs reference master-plan tracks.

P0 — landed / this week

id	title	status	how
AND-1	Release fails-closed; no `/data/local/tmp` in release	✅ **done**	`MainActivity.readBundle` DEBUG-guard + `verifyBundleIntegrity` throws in release
RND-1	`lazy` /thunk memoization	✅ **done**	`native.js` refs short-circuit + `harness/run-lazy.js` in CI gate
L-A0	Re-confirm the probe gates on a fresh emulator after both fixes	☐ 0.2ew	`./scripts/remote.sh android all` against `examples/lumen-probe`; screenshot each gate

P1 — Milestone L-A: Lumen ships on Android

id	title	why (Lumen)	how	ew	deps
L-A1	**Fix Before/After host-layout collapse**	The compare screen "markets the app"; it's the only blocked step in the Android flow	Diagnose why `LumenBeforeAfterView` collapses its Yoga parent — almost certainly a missing `measure` /intrinsic-size or `setHasOverlappingRendering` / `requestLayout` on the custom host view (`host/android/.../views/BeforeAfterView.java` + `CanopyHost.makeView`). Reproduce in `examples/styletest`-style isolation; assert non-zero measured height via `uiautomator dump`. Mirror the built-in-tag pattern, drive `wipeFraction` on the UI thread (pan never crosses the TEA loop).	1.5–2	—
L-A2	**Assemble the real Lumen app**	Turn the probe into the product: the 5 Lumen screens over the live capability set	Build out `/home/quinten/projects/apps/lumen/app/src` against `canopy/native` + capabilities; use side-by-side thumbnails for compare until L-A1 lands, then swap in `Native.BeforeAfter`. Wire navigation (`canopy/navigation` stack).	3–5	L-A1 (compare)
L-A3	**Signed release that boots on device**	No store submission without a signed, R8-shrunk APK/AAB that survives minification of the by-name JNI/JSI lookups	`./gradlew :app:bundleRelease`; author ProGuard keep-rules for `com.canopyhost.modules.*` (reflective `invoke`), JSI registration, ONNX JNI; verify the release AAB boots + restores on device (now that AND-1 makes release fail-closed)	1–1.5	AND-1
L-A4	**Paywall: Play Billing → entitlement**	The "unlock" step; gates restore/export	Wire `BillingModule` (Play Billing one-time product) → persist entitlement via `Storage.Secure` → gate the restore/export action in `update`. Real purchase against a Play internal-test product.	1.5–2	L-A2
L-A5	**Asset + memory budget**	The ~59 MB ORT `.so` + model must fit the AAB size budget; multi-MP restore must not OOM	Decide ORT delivery (in-AAB vs Play Asset Delivery); confirm the ESPCN model packaging; measure peak RSS on a real restore on a Tier-C device, tile/downsample if over budget. Consider dropping unused ORT providers to shrink the `.so`.	1–2	L-A2
L-A6	**E2E: lumen-restore flow on emulator**	Prove the whole flow doesn't regress	Extend `e2e/flows` + `run-e2e.mjs` with the pick→restore→compare→save→share spec selecting by `testID`; run via `e2e/run-matrix.sh`. (The current e2e targets a non-existent app and has never run — this makes it real.)	1	L-A2

P2 — Milestone L-I: Lumen ships on iOS

id	title	why (Lumen)	how	ew	deps
L-I1	**iOS first compile + JSI runtime wired**	No iOS event/render fires without it; the gate for everything iOS	`./scripts/remote.sh ios provision && ios all` on a Mac (today). Expect first-compile errors (ARC/ObjC++, JSI/UIKit signatures, Hermes xcframework ABI). Wire the `jsi::Runtime*` into the host so `canopyEmitEvent` fires.	2–4	a Mac
L-I2	**Capability parity on iOS**	pick/save/share/store/notify must work as on Android	Build + validate each `.mm` module on a simulator/device against the same probe; reach the `lumen-probe` gate set on iOS	2–3	L-I1
L-I3	**RestoreEngine on iOS (Core ML / ANE)**	The product's core, on Apple's NPU	Compile `CanopyRestoreEngineModule.mm`; **fix the dead weak-symbol** `CanopyMakeCoreMLRestoreModule` (declared at `CanopyModuleHost.mm:49`, no definition file, registered by zero paths → silently does nothing); convert/ship the Core ML model	2–3	L-I1
L-I4	**Before/After on iOS + parity vectors**	The compare screen on iOS, provably matching Android	Port the L-A1 compositor to a `UIView` subclass; add a shared platform-neutral test-vector suite so the two hand-written hosts can't silently drift	1.5–2	L-A1, L-I1
L-I5	**StoreKit 2 paywall + TestFlight**	The unlock step on iOS + a shippable build	StoreKit 2 non-consumable → entitlement → gate; signing/provisioning; archive → TestFlight internal	2–3	L-I2
L-I6	**E2E parity on iPhone**	The SAME spec green on both	XCUITest driver for `run-e2e.mjs`; `testID` → `accessibilityIdentifier`; run the L-A6 spec on a device	1	L-A6, L-I2

P3 — both stores / hardening (after L-A and L-I)

Pull from the master plan as needed: Fast Refresh (DEV track — big DX win once shipping), perf head-to-head vs RN (RND), the Hermes/Yoga re-vendor durability gate (RNV), full release CI (CI track building the bundle from source). None block Lumen v1; all make v2 cheaper.

---

4. The single most important next action

Diagnose and fix L-A1 (Before/After host-layout collapse). It is the only blocked step in the Android Lumen flow, it is the screen the product is sold on, and its fix is the design you'll reuse for iOS (L-I4). Everything else on Android either works today or is assembly/release plumbing. Start by reproducing the collapse in isolation on the emulator (`./scripts/remote.sh android all` against a minimal app that mounts only `Native.BeforeAfter`), then instrument the custom view's measured size.

5. What you can build today

The Lumen app's pick / restore / save / share / store / notify paths work on Android right now. Begin assembling the real screens (L-A2) immediately; use side-by-side thumbnails for compare until L-A1 lands. The two correctness fixes from this session (release safety, `lazy`) mean the app you build is both shippable-safe and won't re-diff the whole tree on every frame.